

Lab 02: Numerical and Process-based models

Y Lan Tran (Student ID: 146738902)

2026-08-07

Setup

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Part A: Tracer decay and step size (numerical)

This part shows how the choice of time step affects the accuracy of a numerical tracer decay model, using the forward-Euler results to compare with the exact solution.

```
def decay_numerical(C0, k, dt, t_end):
    """Return arrays of time and concentration for forward-Euler tracer decay."""
    n = int(t_end / dt)
    C = C0
    ts = [0.0]
    Cs = [C0]
    for i in range(n):
        C = C + (-k * C) * dt          # forward difference
        ts.append((i + 1) * dt)
        Cs.append(C)
    return np.array(ts), np.array(Cs)

# The exact answer, for comparison
C0, k, t_end = 100.0, 1.0, 6.0
t_fine = np.linspace(0, t_end, 200)
exact = C0 * np.exp(-k * t_fine)
```

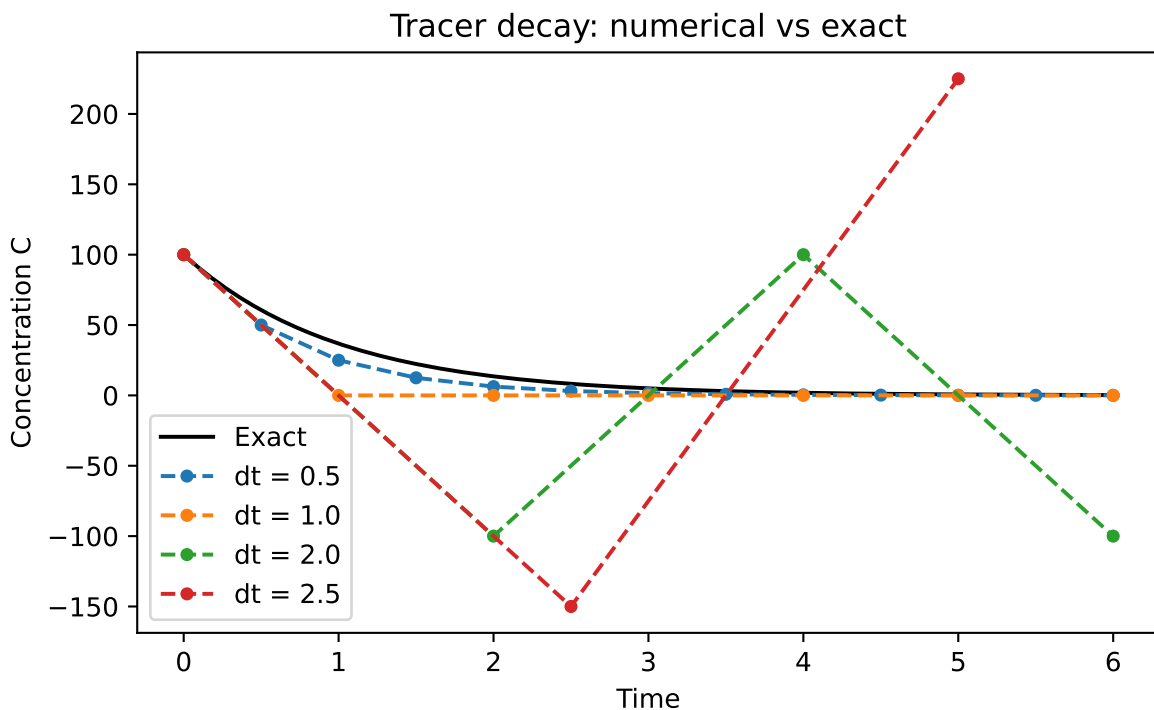
```

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(t_fine, exact, "k-", label="Exact")

for dt in [0.5, 1.0, 2.0, 2.5]:
    ts, Cs = decay_numerical(C0, k, dt, t_end)
    ax.plot(ts, Cs, "o--", markersize=4, label=f"dt = {dt}")

ax.set_xlabel("Time")
ax.set_ylabel("Concentration C")
ax.set_title("Tracer decay: numerical vs exact")
ax.legend()
plt.show()

```



Answer to task A1: The decay rate k controls the speed of concentration decrease. Larger k value means faster decrease or decay and a smaller k causes slower decay.

Answer to task A2: When $dt = 2.0$, the numerical curve is already noticeably different from the exact exponential decay curve.

Answer to task A3: The largest dt value that still gives a non-negative curve is $dt = 1.0$.

Answer to task A4: A large Δt makes each forward-Euler update take a very large step, so the model can remove more tracer than it actually is during one time step. From the update rule $C_{n+1} = C_n(1 - k\Delta t)$, when $k\Delta t > 1$, the factor $(1 - k\Delta t)$ becomes negative, causing the calculated concentration to become negative even though physical concentrations cannot be below zero.

Part B: Reservoir box model (process-based)

```
import numpy as np
import matplotlib.pyplot as plt

# ---- Parameters ----
V = 1.0e6          # lake volume (m3)
Q = 1.0e4          # river inflow (m3 per day)
C_in = 10.0        # incoming concentration (mg/L)

# ---- Time settings ----
dt = 1.0           # time step (days)
days = 200

fig, ax = plt.subplots(figsize=(6, 4))

#Sweep over decay rates
for k in [0.02, 0.05, 0.1, 0.2]:

    # ---- Initial condition ----
    C = 0.0         # clean lake

    # ---- Storage ----
    history = [C]

    # ---- Time loop ----
    for _ in range(days):
        dCdt = (Q * C_in - Q * C - k * V * C) / V  # the rate of change
        C = C + dCdt * dt                          # forward difference
        history.append(C)

    #Plot this curve
    ax.plot(range(days + 1), history, label=f"k = {k}")

# ---- A quick look at the numbers ----
```

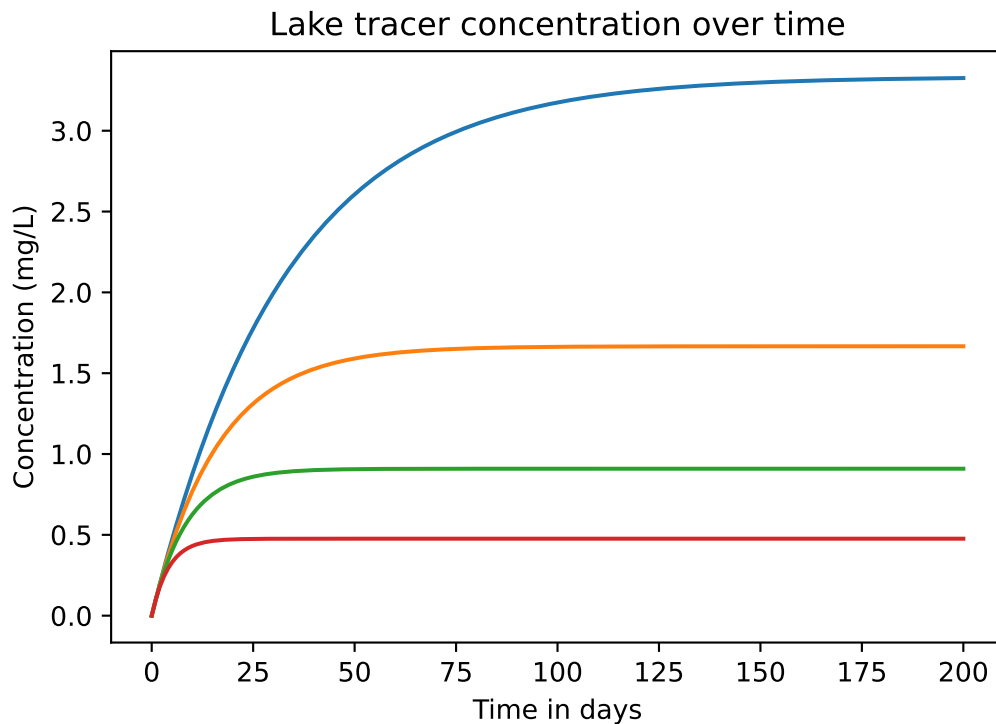
```

print(f"Started at {history[0]:.2f} mg/L")
print(f"After 10 days: {history[10]:.2f} mg/L")
print(f"After 50 days: {history[50]:.2f} mg/L")
print(f"After {days} days: {history[-1]:.2f} mg/L")

ax.set_xlabel("Time in days")
ax.set_ylabel("Concentration (mg/L)")
ax.set_title("Lake tracer concentration over time")
plt.show()

```

Started at 0.00 mg/L
 After 10 days: 0.43 mg/L
 After 50 days: 0.48 mg/L
 After 200 days: 0.48 mg/L



Answer to task B1: The analytical steady-state concentration is 0.909 mg/L and the numerical model reaches 0.909 mg/L after 200 days, (as seen in the graph) confirming that the model is simulating the system correctly.

Answer to task B2: When starting from a polluted lake ($C=20$ mg/L), the concentration decreases over time and approaches the steady-state value of 0.909 mg/L.

Answer to task B3: Doubling the river inflow to $1.0e4$ increases the steady-state concentration to 1.818 mg/L because $C^* = QC_{in}/Q+kV$, so increasing Q increases the numerator more than the denominator, which causes C^* to rise.

Answer to task B4: AS the decay rate k increases, the concentration decreases more quickly and reaches a lower steady-state value.

Reflection

The main issues I had were setting up the files, understanding virtual environments and linking quarto with python. I was surprised at how many programs are needed for one paper. I had to do this multiple times starting from scratch. I found remembering indentations and making sure the loops included all the correct variables tricky and very tedious. I would not spend another hour on this if I did not have to but if I had to I would try to find a real world application and run the simulation with real data.

Checklist before you push

- ☐ Every task has a code cell **and** a short prose answer
- ☐ Every figure has axis labels and a title
- ☐ Your name and student ID are in the YAML header
- ☐ The file renders cleanly (no red error boxes in the HTML output)
- ☐ Filename is `lab-XX-topic.qmd`, not `Untitled.qmd`